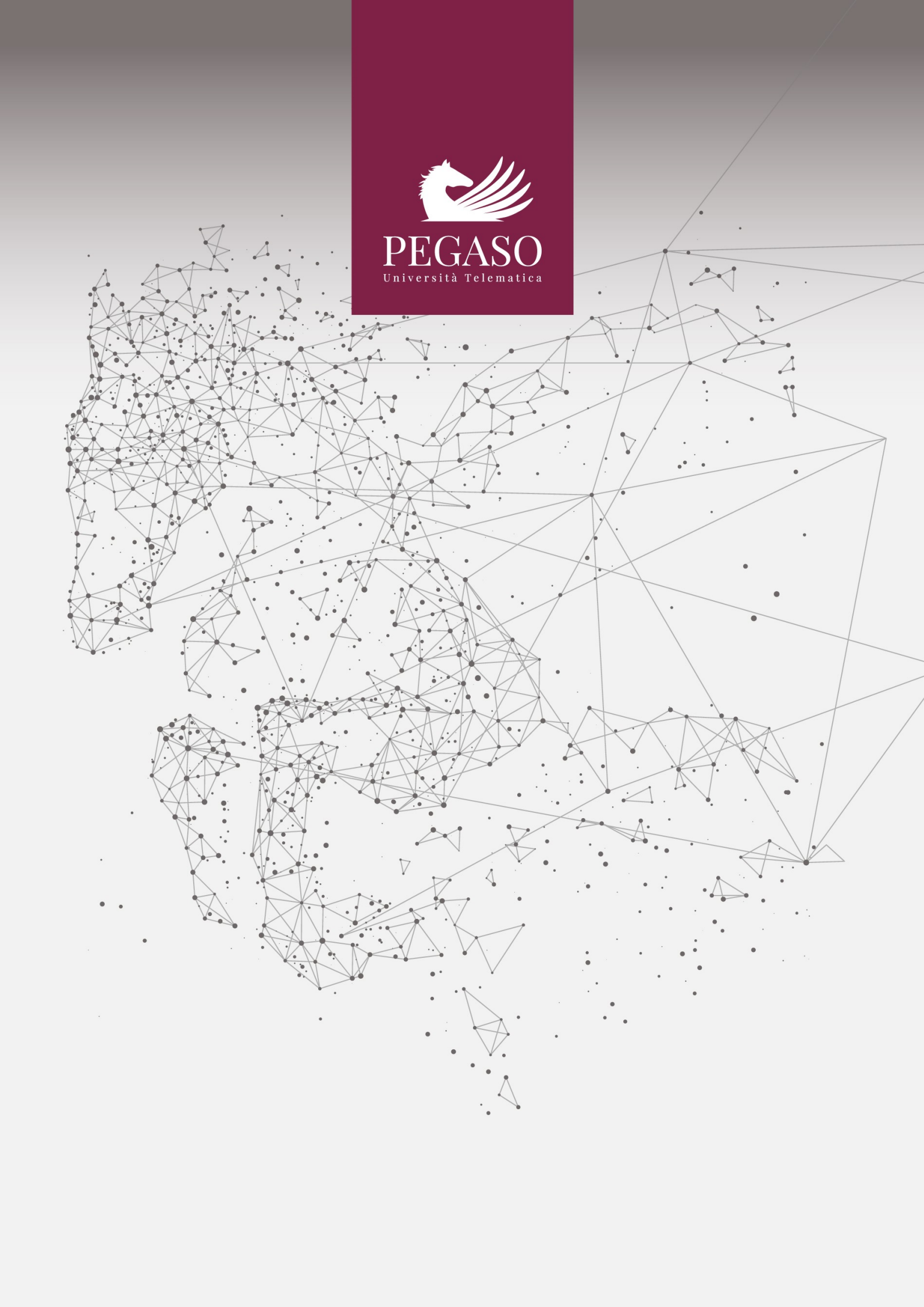

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. ISTRUZIONE ASSEMBLY MIPS SLT 3
2. TIPO DI DATO UNSIGNED 10
3. ISTRUZIONI ARITMETICHE CON OPERANDI UNSIGNED ADDU E SUBU 12
4. ISTRUZIONE DI CONFRONTO TRA OPERANDI UNSIGNED SLTU 13
BIBLIOGRAFIA 16

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Istruzione Assembly MIPS slt

Il Linguaggio Assembly MIPS dispone di istruzioni di salto condizionato che consentono di effettuare il salto all'esecuzione di una istruzione diversa dalla successiva in base al valore di verità di una condizione logica rappresentata dalla relazione di uguaglianza (*branch if equal*) o disuguaglianza (*branch if not equal*) tra due valori.

Per gestire i confronti basati sulla relazione d'ordine *minore*, il Linguaggio Assembly MIPS dispone di istruzioni che consentono di memorizzare in un registro il risultato del confronto tra due valori senza effettuare un salto ad altra istruzione. Il risultato del confronto può essere utilizzato in elaborazioni successive, quindi anche per stabilire se effettuare un salto.

L'istruzione **Assembly MIPS *s*et *o*n *l*ess *t*hen** (imposta a 1 se minore) con Codice operativo **slt** confronta il valore di due operandi producendo un risultato che rappresenta l'esito del confronto, e che viene memorizzato in un registro. La forma sintattica è:

$$\text{slt} \quad \$s0, \ \$t1, \ \$t2$$

l'esecuzione dell'istruzione pone il valore 1 nel registro $\$s0$ se è *vero* che il contenuto del registro $t1 è MINORE del contenuto di $\$t2$, altrimenti (se il risultato del confronto è *falso*) pone il valore 0 in $\$s0$.

Vincoli sintattici:

- Codice Operativo **slt**, ***s*et *o*n *l*ess *t*hen** (imposta a 1 se minore)
- l'**indirizzo del registro DESTINAZIONE** nell'esempio $\$s0$ che dopo l'esecuzione contiene:

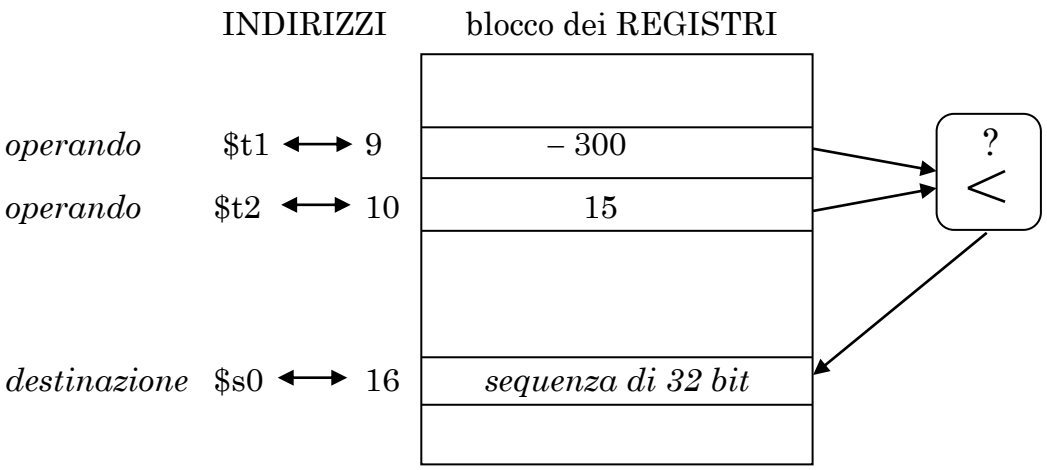
  valore 1 se è *vero* che contenuto di $\$t1 <$ di contenuto di $\$t2$

  valore 0 se è *falso* che contenuto di $\$t1 <$ di contenuto di $\$t2$
- gli **indirizzi degli OPERANDI CON SEGNO da confrontare**, nell'esempio $t1, $t2, espressi in *Notazione in Complemento a 2* seguono separati da virgole l'indirizzo del registro destinazione.

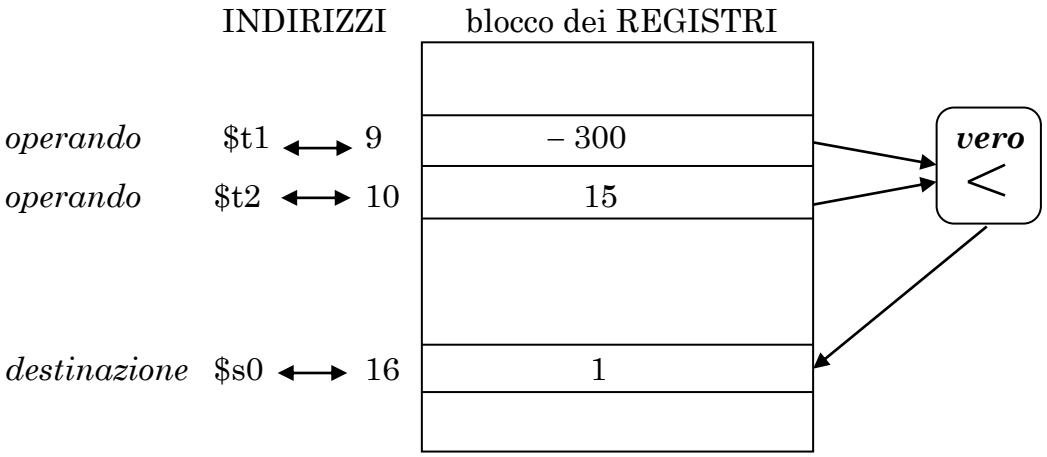
L'esecuzione dell'istruzione ha effetto sul contenuto del registro destinazione, ma lascia immutati i registri degli operandi. Ad esempio, supponendo che il contenuto dei registri **PRIMA dell'esecuzione dell'istruzione** sia il seguente:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



poiché il test di confronto risulta ***vero***, il contenuto nei registri **DOPO l'esecuzione dell'istruzione** è il seguente:



Nelle figure i contenuti dei registri sono indicati con l'usuale notazione decimale. La notazione effettivamente adottata all'interno del computer per la rappresentazione degli interi con segno è la *Notazione in complemento a 2*.

Il confronto tra gli operandi e la determinazione del valore del risultato da scrivere nel registro destinazione è effettuato dalla Unità Aritmetico-Logica (ALU).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'istruzione che traduce in **Linguaggio Macchina MIPS** l'istruzione Assembly ***set on less then*** utilizza:

- Formato di Tipo R,
- Indirizzamento tramite registro,

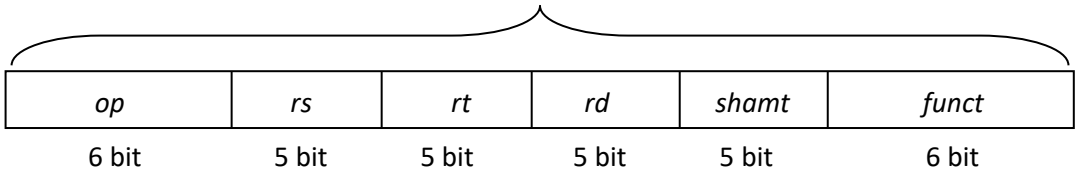
quindi la traduzione e la regola di Indirizzamento sono del tutto analoghe alle istruzioni Aritmetico-Logiche, e vengono qui richiamate.

Il Formato di Tipo R deve la sua denominazione al fatto che gli operandi sono contenuti in registri del processore e gli indirizzi che li individuano sono forniti nei campi dell'istruzione. La lettera R rappresenta, quindi, l'iniziale della parola *Registro*.

Gli indirizzi sono numeri interi positivi rappresentati da 5 bit in Notazione posizionale pesata.

Il Formato di Tipo R suddivide i 32 bit della sequenza che rappresenta l'istruzione in 6 sottosequenze in base alla seguente partizione (concettuale, non fisica) in **CAMPI** di lunghezze e posizioni fissate. Le sottosequenze contenute nei campi sono denotate da nomi simbolici impiegati per descrivere il loro utilizzo nelle istruzioni che usano questo Formato:

FORMATO di Tipo R



op: 6 bit, campo *Codice Operativo* presente in tutte le istruzioni, come nel caso delle istruzioni Aritmetico-Logiche specifica solo il Formato dell'istruzione;

rs: 5 bit, indirizzo del registro che contiene il primo operando;

rt: 5 bit, indirizzo del registro che contiene il secondo operando;

rd: 5 bit, indirizzo del registro che contiene il risultato, detto *registro destinazione*;

shamt: 5 bit, nelle istruzioni di *shift* contiene il numero di posizioni di scorrimento della sequenza (*shift amount*);

funct:6 bit, specifica l'operazione (*function*) relativa all'istruzione.

Il ***Codice Operativo*** è codificato nel campo di 6 bit comune a tutti i Formati. Per l'istruzione ***slt*** il campo *Codice Operativo* contiene il **valore decimale 0**, rappresentato in binario dalla sequenza 000000, che

PEGASO
Università Telematica

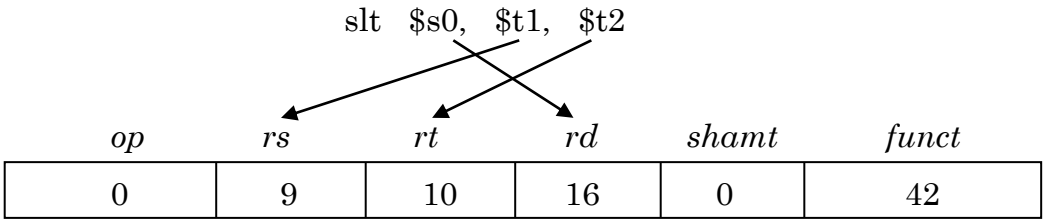
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

indica il Formato. La specifica operazione è rappresentata dai 6 bit del campo ***funct*** (funzione) che contiene il **valore decimale 42**, rappresentato in binario dalla sequenza 101010.

Il campo *shamt* non è utilizzato e viene posto uguale a 0 poiché non esiste la condizione "vuoto" per il contenuto di un registro.

L'ordine degli indirizzi dei registri degli operandi rispecchia quello della relazione per stabilire se il primo operando è *minore* del secondo operando.

La posizione dell'indirizzo del registro destinazione viene invertita rispetto agli operandi come mostrato dal seguente schema:



la cui rappresentazione binaria è:

000000	01001	01010	10000	00000	101010

e quindi la sequenza dei 32 bit in Linguaggio Macchina MIPS è:

00000001001010101000000000101010

Poiché tali istruzioni coinvolgono l'uso dei registri, riportiamo la tabella con gli indirizzi dei registri del processore utilizzati per i dati, indicando gli indirizzi numerici in Notazione decimale e la corrispondente notazione simbolica utilizzata nell'Assembly MIPS:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

simbolo	*indirizzo*	*simbolo*	*indirizzo*	*simbolo*	*indirizzo*
$t0	8	$s0	16	$t8	24
$t1	9	$s1	17	$t9	25
$t2	10	$s2	18		
$t3	11	$s3	19		
$t4	12	$s4	20		
$t5	13	$s5	21		
$t6	14	$s6	22		
$t7	15	$s7	23		

Il Formato di Tipo R viene usato da tutte le istruzioni con una **unica Modalità di Indirizzamento:**

INDIRIZZAMENTO tramite registro

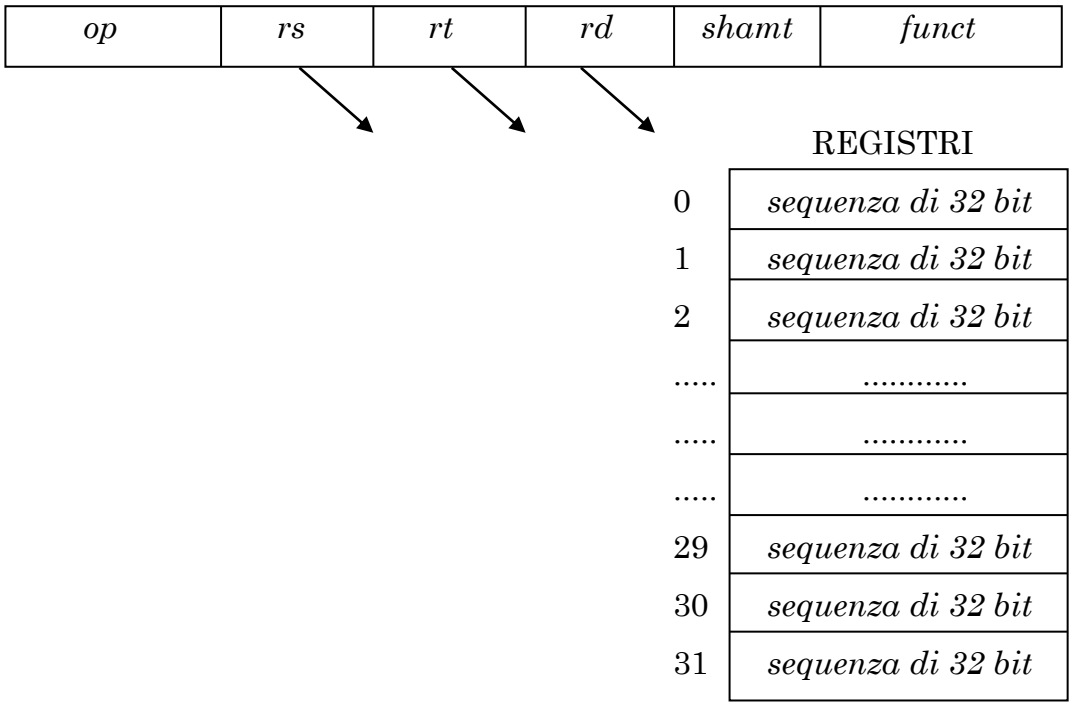
I campi di 5 bit *rs*, *rt*, *rd* del Formato di Tipo R contengono indirizzi di registri del processore in cui sono memorizzati valori di dati coinvolti nella esecuzione dell'istruzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La seguente figura mostra una rappresentazione schematica del significato di tale Modalità di Indirizzamento:

INDIRIZZAMENTO tramite registro



L'istruzione *slt* viene utilizzata insieme alle istruzioni di salto condizionato per gestire le condizioni logiche nella traduzione delle istruzioni di controllo del flusso di esecuzione dei Linguaggi ad Alto Livello.

ESEMPIO

Istruzioni con variabili intere:

```
if ( a < b ) {
   min = a ;
}
else {
   min= b ;
}
a = 0 ;
```

***salta** a ELSE*
se a < b è ***falsa***

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Come *condizione logica* dell'istruzione *if-else* viene valuta la relazione a < b. Se la condizione è *vera* si effettua l'assegnamento min = a, se è *falsa* si effettua l'assegnamento min = b. In ogni caso l'istruzione successiva è l'assegnamento a = 0.

Assegnando alle variabili i seguenti registri:

a	b	min
$t1	$t2	$s0

una traduzione in Assembly MIPS è data dalle istruzioni:

```
        slt   $t5, $t1, $t2          # pone il contenuto di $t5 = 1
                                     # se a < b è vera, altrimenti lo pone uguale a 0
        beq   $t5, $zero, ELSE       #      salta a ELSE
                                     # se contenuto di $t5 == 0 è vera,        ⟺
                                     # se a < b è falsa
        addi  $s0, $t1, 0            #  min = a
        j  END_IF                    #  esce da if-else
ELSE:   addi  $s0, $t2, 0            #  min = b
END_IF: add   $t1, $zero, $zero      #  a = 0.
```

Si può notare che, per mantenere l'ordine dei casi con *else* come secondo, **nella traduzione Assembly** viene utilizzata l'istruzione di salto condizionato ***beq*** per saltare a *else* se il registro $t5 contiene il valore 0 che corrisponde al risultato del confronto a<b *falso*.

Quindi la decisione se effettuare il salto al caso *else* avviene in due passi:

- prima l'istruzione *slt* esegue il confronto in base alla relazione d'ordine *minore* e memorizza il risultato nel registro $t5,
- poi l'istruzione *beq* effettua il salto a *else* se il contenuto di $t5 è 0 (che equivale al risultato del confronto *falso*) altrimenti si continua eseguendo la parte *if*,

il blocco *if* termina con una istruzione *j* per saltare alla esecuzione della prossima istruzione evitando l'esecuzione del blocco *else*.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Tipo di dato unsigned

Lo sviluppo di particolari applicazioni può richiede l'utilizzazione di numeri interi senza segno, detti di Tipo *unsigned*.

All'interno del computer gli interi con segno sono rappresentati in base alla *Notazione in complemento a* 2, che divide l'intervallo dei valori rappresentabili in una metà positiva ed una negativa. In questa notazione il bit più significativo è associato ad una potenza negativa.

Per la rappresentazione degli interi *unsigned* viene invece usata la *Notazione posizionale pesata*. In questa notazione tutti i bit sono associati a potenze positive.

ESEMPIO

Sequenze di 8 bit (un Byte).

In base alle regole per il calcolo del valore nelle due Notazioni, il bit più significativo determina la differenza tra i valori rappresentati nel caso di numeri negativi:

11111111 {

$= -2^7 + 127 = -128 + 127 = -1$

Notazione in complemento a 2 (con segno)

$= 2^7 + 127 = 128 + 127 = 255$

Notazione posizionale pesata (*unsigned*)

}

Sempre in base alle regole per il calcolo del valore nelle due Notazioni, gli intervalli dei valori rappresentabili con N bit sono:

Notazione in complemento a 2: $[\ -2^{N-1},\ 2^{N-1}-1\ ]$

Notazione posizionale pesata: $[\ 0,\ \ 2^{N}-1\ ]$

quindi, per N=8 si hanno gli intervalli $[\ -128,\ 127\ ]$ e $[\ 0,\ 255\ ]$.

I Linguaggi C e C++ dispongono di un Tipo elementare *unsigned* la cui dichiarazione è ottenuta mediante il termine *unsigned* anteposto ai Tipi elementari dei numeri con segno *char*, *int*, *long*.

Il Linguaggio Java non dispone di un Tipo elementare *unsigned* per i Tipi elementari dei numeri con segno *byte, short, int, long*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per soddisfare la possibilità di elaborare anche questo Tipo di dato, a partire dalla versione Java SE 8 la classe *Integer* è stata dotata di metodi per la gestione di operazioni tra interi considerati come *unsigned*.

Il **Linguaggio Assembly** MIPS comprende istruzioni con operandi *unsigned*. Le regole sintattiche sono analoghe alle istruzioni con operandi con segno e il Codice Operativo è uguale con l'aggiunta del suffisso "u" finale.

La traduzione in **Linguaggio Macchina** utilizza Formato e Modalità di Indirizzamento analoghi alle istruzioni con operandi con segno.

Le istruzioni con operandi *unsigned* contenuti nei registri del processore utilizzano sequenze di 32 bit per la rappresentazione dei valori.

Per il Tipo di dato *unsigned* la rappresentazione interna è in *Notazione posizionale pesata*. Con sequenze di 32 bit si ha che il bit più significativo è associato al peso positivo 2^{31}, e l'intervallo dei valori rappresentabili dato da $[0, 2^{32} - 1]$ contiene circa 4 miliardi di valori tutti positivi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Istruzioni aritmetiche con operandi unsigned addu e subu

Il Linguaggio Assembly MIPS dispone di istruzioni Aritmetiche per operandi *unsigned* contenuti nei registri del processore.

Le istruzioni **Assembly** MIPS ADDIZIONE e SOTTRAZIONE per operandi *unsigned* hanno, rispettivamente, Codice Operativo **addu** e **subu**

Le regole sintattiche ed il significato sono analoghi alle istruzioni *add* e *sub*:

```
addu  $s0,  $t1,  $t2
subu  $s0,  $t1,  $t2
```

con $t1, $t2 indirizzi dei registri che contengono gli operandi *unsigned* e $s0 indirizzo del registro destinazione che contiene il risultato dell'operazione.

Anche le corrispondenti istruzioni in **Linguaggio Macchina** hanno Formato di Tipo R e Indirizzamento tramite registro analoghi alle istruzioni *add* e *sub* con operandi con segno.

Cambia solo il valore dei 6 bit del campo ***funct*** (funzione) che specifica l'operazione, riportato nella seguente tabella per le due istruzioni in decimale ed in binario:

	6 bit del campo ***funct***	
istruzione	decimale	binario
addu	33	100001
subu	35	100011

Le operazioni di somma e sottrazione si eseguono sempre sui bit che occupano le posizioni corrispondenti:

- addizionando il bit di riporto con i due bit degli operandi,
- generando il bit del risultato ed il bit di riporto sulla posizione successiva.

Cambia il controllo dell'**Overflow** in quanto per operazioni tra dati *unsigned* non è più valida la regola basata sul valore degli ultimi due riporti calcolati, utilizzata nella Notazione in complemento a 2.

OSSERVAZIONE

Le istruzioni logiche *and* e *or*, che effettuano le operazioni AND e OR su sequenze contenute in registri, non hanno corrispondenti istruzioni per operandi *unsigned*, perché le operazioni vengono eseguite *bit a bit*, quindi solo su sequenze considerate senza segno.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Istruzione di confronto tra operandi unsigned sltu

L'istruzione **Assembly** MIPS *set on less then* per operandi *unsigned* ha Codice Operativo ***sltu***.
Le regole sintattiche ed il significato sono analoghi all'istruzione *slt*:

```
sltu   $s0,   $t1,   $t2
```

Anche la corrispondente istruzione in **Linguaggio Macchina** ha Formato di Tipo R e Modalità di Indirizzamento tramite registro analoghi all'istruzione *slt* con operandi interi con segno.

Cambia solo il valore dei 6 bit del campo ***funct*** (funzione) che contiene il **valore decimale 43**, rappresentato in binario dalla sequenza 101011.

Le due istruzioni *slt* e *sltu* danno risultati diversi quando sono coinvolti numeri negativi in quanto il bit più significativo associato a pesi diversi nelle notazioni con segno e senza segno determina valori diversi.

In particolare, per i numeri con segno nella Notazione in complemento a 2, i valori rappresentati dalle sequenze con bit più significativo uguale a 1 (numeri negativi) risultano minori di tutti i valori rappresentati dalle sequenze che hanno tale bit uguale a 0 (numeri positivi). Invece, la relazione si inverte quando tali sequenze sono considerate *unsigned*.
Diamo un esempio di questo tipo di risultato considerando le due seguenti istruzioni:

```
slt   $s0,   $t1,   $t2
sltu   $s0,   $t1,   $t2
```

Supponiamo che il contenuto dei registri operandi sia:

contenuto del registro di indirizzo **$t1**

11111111111111111111111111111111

che rappresenta il valore −1 in Notazione in complemento a 2 (con segno), ed il valore $2^{32}-1$ pari a circa 4 miliardi in Notazione posizionale pesata (senza segno):

contenuto del registro di indirizzo **$t2**

00000000000000000000000000000001

che rappresenta sempre il valore 1 in entrambe le Notazioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato della esecuzione delle due istruzioni è:

```
slt   $s0,   $t1,   $t2     # il registro di indirizzo $s0 contiene 1
                            # poiché  –1 < 1  è vera
sltu  $s0,   $t1,   $t2     # il registro di indirizzo $s0 contiene 0
                            # poiché  2^32–1  < 1  è falsa.
```

Le seguenti Tabelle riassumono per il sottoinsieme di istruzioni presentate le regole sintattiche del Linguaggio Assembly e del Liguaggio Macchina MIPS, e la relativa Modalità di Indirizzamento.

I valori dei campi del Formato in Linguaggio Macchina sono espressi mediante la notazione decimale per i valori numerici, e la notazione simbolica per gli indirizzi dei registri del processore.

		6 bit	5 bit	5 bit	5 bit	5 bit	6 bit	esempio
istruzione	*Tipo*	*op*	*rs*	*rt*	*rd*	*shamt*	*funct*	
add	R	0	$s1	$s2	$t0	0	32	add $t0, $s1, $s2
sub	R	0	$s1	$s2	$t0	0	34	sub $t0, $s1, $s2
and	R	0	$s1	$s2	$t0	0	36	and $t0, $s1, $s2
or	R	0	$s1	$s2	$t0	0	37	or $t0, $s1, $s2
slt	R	0	$s1	$s2	$t0	0	42	slt $t0, $s1, $s2
sll	R	0	0	$s2	$t0	7	0	sll $t0, $s2, 7
srl	R	0	0	$s2	$t0	7	2	srl $t0, $s2, 7
lw	I	35	$s3	$t0	–9			lw $t0, –9 ($s3)
sw	I	43	$s3	$t0	–9			sw $t0, –9 ($s3)
beq	I	4	$t0	$s3	istruzioni da saltare			beq $t0, $s3, *nome*
bne	I	5	$t0	$s3	istruzioni da saltare			bne $t0, $s3, *nome*
j	J	2	indirizzo di salto incompleto					j *nome*
jal	J	3	indirizzo di salto incompleto					jal *nome*
jr	R	0	$ra	0	0	0	8	jr $ra
addu	R	0	$s1	$s2	$t0	0	33	addu $t0, $s1, $s2
subu	R	0	$s1	$s2	$t0	0	35	subu $t0, $s1, $s2
sltu	R	0	$s1	$s2	$t0	0	43	sltu $t0, $s1, $s2
addi	I	8	$s3	$t0	–9			addi $t0, $s3,–9

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Istruzione	*Formato Tipo*	*Modalità di Indirizzamento*
add	R	tramite registro
sub	R	tramite registro
and	R	tramite registro
or	R	tramite registro
slt	R	tramite registro
sll	R	tramite registro
srl	R	tramite registro
lw	I	tramite Base e Offset
sw	I	tramite Base e Offset
beq	I	relativo al *Program Counter*
bne	I	relativo al *Program Counter*
j	J	pseudodiretto
jal	J	pseudodiretto
jr	R	tramite registro
addu	R	tramite registro
subu	R	tramite registro
sltu	R	tramite registro
addi	I	immediato

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.
- ORACLE Java Documentation, The Java Tutorials, Primitive Data Types
  https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).